



Figure 14.1: The general structure of an autoencoder, mapping an input $\mathbf{x}$ to an output (called reconstruction) $\mathbf{r}$ through an internal representation or code $\mathbf{h}$. The autoencoder has two components: the encoder f (mapping $\mathbf{x}$ to $\mathbf{h}$) and the decoder g (mapping $\mathbf{h}$ to $\mathbf{r}$).

to the activations on the reconstructed input. Recirculation is regarded as more biologically plausible than back-propagation but is rarely used for machine learning applications.

14.1 Undercomplete Autoencoders

Copying the input to the output may sound useless, but we are typically not interested in the output of the decoder. Instead, we hope that training the autoencoder to perform the input copying task will result in $\mathbf{h}$ taking on useful properties.

One way to obtain useful features from the autoencoder is to constrain $\mathbf{h}$ to have a smaller dimension than $\mathbf{x}$. An autoencoder whose code dimension is less than the input dimension is called **undercomplete**. Learning an undercomplete representation forces the autoencoder to capture the most salient features of the training data.

The learning process is described simply as minimizing a loss function

$$L(\mathbf{x}, g(f(\mathbf{x}))), \quad (14.1)$$

where L is a loss function penalizing $g(f(\mathbf{x}))$ for being dissimilar from $\mathbf{x}$, such as the mean squared error.

When the decoder is linear and L is the mean squared error, an undercomplete autoencoder learns to span the same subspace as PCA. In this case, an autoencoder trained to perform the copying task has learned the principal subspace of the training data as a side effect.

Autoencoders with nonlinear encoder functions f and nonlinear decoder functions g can thus learn a more powerful nonlinear generalization of PCA. Unfortunately,

Chapter 14 Autoencoders

An **autoencoder** is a neural network that is trained to attempt to copy its input to its output. Internally, it has a hidden layer $\mathbf{h}$ that describes a **code** used to represent the input. The network may be viewed as consisting of two parts: an encoder function $\mathbf{h} = f(\mathbf{x})$ and a decoder that produces a reconstruction $\mathbf{r} = g(\mathbf{h})$. This architecture is presented in figure 14.1. If an autoencoder succeeds in simply learning to set $g(f(\mathbf{x})) = \mathbf{x}$ everywhere, then it is not especially useful. Instead, autoencoders are designed to be unable to learn to copy perfectly. Usually they are restricted in ways that allow them to copy only approximately, and to copy only input that resembles the training data. Because the model is forced to prioritize which aspects of the input should be copied, it often learns useful properties of the data.

Modern autoencoders have generalized the idea of an encoder and a decoder beyond deterministic functions to stochastic mappings $p_{\text{encoder}}(\mathbf{h} | \mathbf{x})$ and $p_{\text{decoder}}(\mathbf{x} | \mathbf{h})$.

The idea of autoencoders has been part of the historical landscape of neural networks for decades (LeCun, 1987; Bourlard and Kamp, 1988; Hinton and Zemel, 1994). Traditionally, autoencoders were used for dimensionality reduction or feature learning. Recently, theoretical connections between autoencoders and latent variable models have brought autoencoders to the forefront of generative modeling, as we will see in chapter 20. Autoencoders may be thought of as being a special case of feedforward networks and may be trained with all the same techniques, typically minibatch gradient descent following gradients computed by back-propagation. Unlike general feedforward networks, autoencoders may also be trained using **recirculation** (Hinton and McClelland, 1988), a learning algorithm based on comparing the activations of the network on the original input

nately, if the encoder and decoder are allowed too much capacity, the autoencoder can learn to perform the copying task without extracting useful information about the distribution of the data. Theoretically, one could imagine that an autoencoder with a one-dimensional code but a very powerful nonlinear encoder could learn to represent each training example $\mathbf{x}^{(i)}$ with the code i . The decoder could learn to map these integer indices back to the values of specific training examples. This specific scenario does not occur in practice, but it illustrates clearly that an autoencoder trained to perform the copying task can fail to learn anything useful about the dataset if the capacity of the autoencoder is allowed to become too great.

14.2 Regularized Autoencoders

Undercomplete autoencoders, with code dimension less than the input dimension, can learn the most salient features of the data distribution. We have seen that these autoencoders fail to learn anything useful if the encoder and decoder are given too much capacity.

A similar problem occurs if the hidden code is allowed to have dimension equal to the input, and in the **overcomplete** case in which the hidden code has dimension greater than the input. In these cases, even a linear encoder and a linear decoder can learn to copy the input to the output without learning anything useful about the data distribution.

Ideally, one could train any architecture of autoencoder successfully, choosing the code dimension and the capacity of the encoder and decoder based on the complexity of distribution to be modeled. Regularized autoencoders provide the ability to do so. Rather than limiting the model capacity by keeping the encoder and decoder shallow and the code size small, regularized autoencoders use a loss function that encourages the model to have other properties besides the ability to copy its input to its output. These other properties include sparsity of the representation, smallness of the derivative of the representation, and robustness to noise or to missing inputs. A regularized autoencoder can be nonlinear and overcomplete but still learn something useful about the data distribution, even if the model capacity is great enough to learn a trivial identity function.

In addition to the methods described here, which are most naturally interpreted as regularized autoencoders, nearly any generative model with latent variables and equipped with an inference procedure (for computing latent representations given input) may be viewed as a particular form of autoencoder. Two generative modeling approaches that emphasize this connection with autoencoders are the descendants of the Helmholtz machine (Hinton *et al.*, 1995b), such as the variational

autoencoder (section 20.10.3) and the generative stochastic networks (section 20.12). These models naturally learn high-capacity, overcomplete encodings of the input and do not require regularization for these encodings to be useful. Their encodings are naturally useful because the models were trained to approximately maximize the probability of the training data rather than to copy the input to the output.

14.2.1 Sparse Autoencoders

A sparse autoencoder is simply an autoencoder whose training criterion involves a sparsity penalty $\Omega(\mathbf{h})$ on the code layer $\mathbf{h}$, in addition to the reconstruction error:

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h}), \quad (14.2)$$

where $g(\mathbf{h})$ is the decoder output, and typically we have $\mathbf{h} = f(\mathbf{x})$, the encoder output.

Sparse autoencoders are typically used to learn features for another task, such as classification. An autoencoder that has been regularized to be sparse must respond to unique statistical features of the dataset it has been trained on, rather than simply acting as an identity function. In this way, training to perform the copying task with a sparsity penalty can yield a model that has learned useful features as a byproduct.

We can think of the penalty $\Omega(\mathbf{h})$ simply as a regularizer term added to a feedforward network whose primary task is to copy the input to the output (unsupervised learning objective) and possibly also perform some supervised task (with a supervised learning objective) that depends on these sparse features. Unlike other regularizers, such as weight decay, there is not a straightforward Bayesian interpretation to this regularizer. As described in section 5.6.1, training with weight decay and other regularization penalties can be interpreted as a MAP approximation to Bayesian inference, with the added regularizing penalty corresponding to a prior probability distribution over the model parameters. In this view, regularized maximum likelihood corresponds to maximizing $p(\boldsymbol{\theta} | \mathbf{x})$, which is equivalent to maximizing $\log p(\mathbf{x} | \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})$. The $\log p(\mathbf{x} | \boldsymbol{\theta})$ term is the usual data log-likelihood term, and the $\log p(\boldsymbol{\theta})$ term, the log-prior over parameters, incorporates the preference over particular values of $\boldsymbol{\theta}$. This view is described in section 5.6. Regularized autoencoders defy such an interpretation because the regularizer depends on the data and is therefore by definition not a prior in the formal sense of the word. We can still think of these regularization terms as implicitly expressing a preference over functions.

Rather than thinking of the sparsity penalty as a regularizer for the copying task, we can think of the entire sparse autoencoder framework as approximating

maximum likelihood training of a generative model that has latent variables. Suppose we have a model with visible variables $\mathbf{x}$ and latent variables $\mathbf{h}$, with an explicit joint distribution $p_{\text{model}}(\mathbf{x}, \mathbf{h}) = p_{\text{model}}(\mathbf{h})p_{\text{model}}(\mathbf{x} | \mathbf{h})$. We refer to $p_{\text{model}}(\mathbf{h})$ as the model's prior distribution over the latent variables, representing the model's beliefs prior to seeing $\mathbf{x}$. This is different from the way we have previously used the word "prior," to refer to the distribution $p(\boldsymbol{\theta})$ encoding our beliefs about the model's parameters before we have seen the training data. The log-likelihood can be decomposed as

$$\log p_{\text{model}}(\mathbf{x}) = \log \sum_{\mathbf{h}} p_{\text{model}}(\mathbf{h}, \mathbf{x}). \quad (14.3)$$

We can think of the autoencoder as approximating this sum with a point estimate for just one highly likely value for $\mathbf{h}$. This is similar to the sparse coding generative model (section 13.4), but with $\mathbf{h}$ being the output of the parametric encoder rather than the result of an optimization that infers the most likely $\mathbf{h}$. From this point of view, with this chosen $\mathbf{h}$, we are maximizing

$$\log p_{\text{model}}(\mathbf{h}, \mathbf{x}) = \log p_{\text{model}}(\mathbf{h}) + \log p_{\text{model}}(\mathbf{x} | \mathbf{h}). \quad (14.4)$$

The $\log p_{\text{model}}(\mathbf{h})$ term can be sparsity inducing. For example, the Laplace prior,

$$p_{\text{model}}(h_i) = \frac{\lambda}{2} e^{-\lambda|h_i|}, \quad (14.5)$$

corresponds to an absolute value sparsity penalty. Expressing the log-prior as an absolute value penalty, we obtain

$$\begin{aligned} \Omega(\mathbf{h}) &= \lambda \sum_i |h_i|, & (14.6) \\ -\log p_{\text{model}}(\mathbf{h}) &= \sum_i \left(\lambda|h_i| - \log \frac{\lambda}{2} \right) = \Omega(\mathbf{h}) + \text{const}, & (14.7) \end{aligned}$$

where the constant term depends only on λ and not $\mathbf{h}$. We typically treat λ as a hyperparameter and discard the constant term since it does not affect the parameter learning. Other priors, such as the Student t prior, can also induce sparsity. From this point of view of sparsity as resulting from the effect of $p_{\text{model}}(\mathbf{h})$ on approximate maximum likelihood learning, the sparsity penalty is not a regularization term at all. It is just a consequence of the model's distribution over its latent variables. This view provides a different motivation for training an autoencoder: it is a way of approximately training a generative model. It also provides a different reason for

why the features learned by the autoencoder are useful: they describe the latent variables that explain the input.

Early work on sparse autoencoders (Ranzato *et al.*, 2007a, 2008) explored various forms of sparsity and proposed a connection between the sparsity penalty and the $\log Z$ term that arises when applying maximum likelihood to an undirected probabilistic model $p(\mathbf{x}) = \frac{1}{Z} \tilde{p}(\mathbf{x})$. The idea is that minimizing $\log Z$ prevents a probabilistic model from having high probability everywhere, and imposing sparsity on an autoencoder prevents the autoencoder from having low reconstruction error everywhere. In this case, the connection is on the level of an intuitive understanding of a general mechanism rather than a mathematical correspondence. The interpretation of the sparsity penalty as corresponding to $\log p_{\text{model}}(\mathbf{h})$ in a directed model $p_{\text{model}}(\mathbf{h})p_{\text{model}}(\mathbf{x} | \mathbf{h})$ is more mathematically straightforward.

One way to achieve *actual zeros* in $\mathbf{h}$ for sparse (and denoising) autoencoders was introduced in Glorot *et al.* (2011b). The idea is to use rectified linear units to produce the code layer. With a prior that actually pushes the representations to zero (like the absolute value penalty), one can thus indirectly control the average number of zeros in the representation.

14.2.2 Denoising Autoencoders

Rather than adding a penalty Ω to the cost function, we can obtain an autoencoder that learns something useful by changing the reconstruction error term of the cost function.

Traditionally, autoencoders minimize some function

$$L(\mathbf{x}, g(f(\mathbf{x}))), \quad (14.8)$$

where L is a loss function penalizing $g(f(\mathbf{x}))$ for being dissimilar from $\mathbf{x}$, such as the L^2 norm of their difference. This encourages $g \circ f$ to learn to be merely an identity function if they have the capacity to do so.

A **denoising autoencoder** (DAE) instead minimizes

$$L(\mathbf{x}, g(f(\tilde{\mathbf{x}}))), \quad (14.9)$$

where $\tilde{\mathbf{x}}$ is a copy of $\mathbf{x}$ that has been corrupted by some form of noise. Denoising autoencoders must therefore undo this corruption rather than simply copying their input.

Denoising training forces f and g to implicitly learn the structure of $p_{\text{data}}(\mathbf{x})$, as shown by Alain and Bengio (2013) and Bengio *et al.* (2013c). Denoising

autoencoders thus provide yet another example of how useful properties can emerge as a byproduct of minimizing reconstruction error. They are also an example of how overcomplete, high-capacity models may be used as autoencoders as long as care is taken to prevent them from learning the identity function. Denoising autoencoders are presented in more detail in section 14.5.

14.2.3 Regularizing by Penalizing Derivatives

Another strategy for regularizing an autoencoder is to use a penalty Ω , as in sparse autoencoders,

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h}, \mathbf{x}), \quad (14.10)$$

but with a different form of Ω :

$$\Omega(\mathbf{h}, \mathbf{x}) = \lambda \sum_i \|\nabla_{\mathbf{x}} h_i\|^2. \quad (14.11)$$

This forces the model to learn a function that does not change much when $\mathbf{x}$ changes slightly. Because this penalty is applied only at training examples, it forces the autoencoder to learn features that capture information about the training distribution.

An autoencoder regularized in this way is called a **contractive autoencoder**, or CAE. This approach has theoretical connections to denoising autoencoders, manifold learning, and probabilistic modeling. The CAE is described in more detail in section 14.7.

14.3 Representational Power, Layer Size and Depth

Autoencoders are often trained with only a single layer encoder and a single layer decoder. However, this is not a requirement. In fact, using deep encoders and decoders offers many advantages.

Recall from section 6.4.1 that there are many advantages to depth in a feedforward network. Because autoencoders are feedforward networks, these advantages also apply to autoencoders. Moreover, the encoder is itself a feedforward network, as is the decoder, so each of these components of the autoencoder can individually benefit from depth.

One major advantage of nontrivial depth is that the universal approximator theorem guarantees that a feedforward neural network with at least one hidden layer can represent an approximation of any function (within a broad class) to an